

Arrays in Java

14 August 2026 05:55

Java arrays are objects that store multiple elements of the same data type in a single contiguous memory block.
Arrays allow us to group similar data together and access them using an index.

Arrays in Java

Arrays in Java are index based, meaning the first element is at index 0 and the second at index 1 and so on. In Java an array is considered an object of dynamically created class. All arrays in Java:

- Derived from the Object class.
- Array can store primitive values or objects and Java supports both single dimensional and multi-dimensional array.

Single-dimensional array in Java

A single dimensional array in Java is a linear collection of elements of the same data type. It can be declared and instantiated using the following syntax:

```
datatype[] arr; or
datatype []arr; or
datatype arr[];
```

Syntax of array instantiation:

```
Arr = new datatype[size];
```

The **new** keyword allocated the memory dynamically(at runtime) on the heap memory area.

```
package MyArrays;
import java.util.*;
```

```
/**
 * @author (your name)
 * @version (a version number or a date)
 */
public class SDArrayDemo
{
    public static Scanner sc = new Scanner(System.in);
    public static void main(String[] args){
        System.out.print("\n");
        int[] Arr1, i; //all the variable will become an array
        int []Arr2, j; //all the variable will become an array
        int Arr3[], k; //all the variables will not become an array
        Arr1 = new int[5];
        Arr2 = new int[5];
        Arr3 = new int[5];

        // i = 0; // this type of assgnment is not allowed in the array
        // j = 0; //j is an array
        k = 0;
```

```

//Declaring and allocating memory for the array.
int Arr4[] = new int[5];

Arr1[0] = 10;
Arr1[1] = 20;
Arr1[2] = 30;
Arr1[3] = 40;
Arr1[4] = 50;
System.out.print("\nArr1[");
for(k = 0; k < Arr1.length - 1; k++){
    System.out.print(Arr1[k]+", ");
}
System.out.print(Arr1[k+"]\n");

String []month = {"Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul",
    "Aug", "Sep", "Oct", "Nov", "Dec"};
System.out.print("\nMonths: ");
for(String m : month){ //for each loop is mainly used for Arrays
and Collection
    System.out.print(m+" ");
}
System.out.print("\n");

SDArrayUtility.printSDArray(new int[]{100,200,300,400}, "Anonymous
Array[]: ");

//Don't use in school examination
System.out.print("\nArr1: " + java.util.Arrays.toString(Arr1));

}
}

```

Output:

```

BlueJ: Terminal Window - Kritin11
Options
Arr1[10, 20, 30, 40, 50]
Months: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
Anonymous Array[]:
[100, 200, 300, 400]
Arr1: [10, 20, 30, 40, 50]

```

ArrayIndexOutOfBoundsException

In Java, the `ArrayIndexOutOfBoundsException` is a runtime exception. Thrown by the Java Virtual Machine (JVM) when attempting to access an invalid index of an array. This exception occurs if the index is negative, Equal to the size of the array or greater than the size of the array while traversing the array.

Homework:

1: Ordered Insertion in an Array (Without Sorting)

Write a program in Java to input N integers into an array in ascending order without using any sorting algorithm.
The user may enter the numbers in any order. As each number is entered, insert it into its correct position in the array by shifting the larger elements one position to the right.
Display the array after every insertion and finally print the complete sorted array.

Sample Input

Enter number of elements: 6

Enter number: 20

Enter number: 10

Enter number: 15

Enter number: 5

Enter number: 18

Enter number: 12

Output

After inserting 20

[20]

After inserting 10

[10, 20]

After inserting 15

[10, 15, 20]

After inserting 5

[5, 10, 15, 20]

After inserting 18

[5, 10, 15, 18, 20]

After inserting 12

[5, 10, 12, 15, 18, 20]

Restrictions

- Do not use `Arrays.sort()`.
- Do not use any Collection classes.
- Insert each element at its proper position while taking input.

2: Second Highest Distinct Element

Write a program to input N integers into an array and determine the second highest distinct element without sorting the array.
If the second highest distinct element does not exist, display an appropriate message.

Sample Input 1

8

15 22 11 45 45 10 39 22

Output

Highest = 45

Second Highest = 39

Sample Input 2

5

20 20 20 20 20

Output

Second Highest Element does not exist.

Restrictions

- Do not sort the array.
- Traverse the array only once after input.
- Do not use an additional array.

3: Insert an Element at a Given Position

Write a program to create an array of N elements.

Input a new element X and a position P (0-based index). Insert the element at the specified position by shifting the remaining elements to the right.

If the array is already full, display "Insertion not possible."

Example

Original Array

[10, 20, 30, 40, 50]

Enter element : 25

Enter position : 2

Output

Array after insertion

[10, 20, 25, 30, 40, 50]

Restrictions

- Do not create another array.
- Perform insertion using shifting only.

4: Maximum Difference

Write a program to input N integers into an array.

Find the largest positive difference between any two elements such that the larger element appears after the smaller element.

If no such pair exists, print 0.

Example

Input

9

7 1 5 3 6 4 8 2 10

Output

Largest Difference = 9

(10 - 1)

Example

Input

5

10 9 8 7 6

Output

Largest Difference = 0

Restrictions

- Do not sort the array.
- Use only one traversal after taking input.

5: Array Rotation Challenge

Write a program to input N integers into an array.

Input an integer K and rotate the array to the right by K positions.

If K is greater than N, rotate by the effective number of positions.

Example

Input

Array

10 20 30 40 50 60

K = 2

Output

50 60 10 20 30 40

Restrictions

- Do not use another array of size N.
- Perform the rotation manually.

6: Longest Consecutive Increasing Sequence

Write a program to input N integers into an array.

Determine the longest consecutive increasing sequence present in the array.

Print

- Starting index
- Ending index
- Length of the sequence
- Elements of the sequence

Example

Input

12

5 8 10 15 3 7 9 11 13 2 4 6

Output

Longest Consecutive Increasing Sequence

7 9 11 13

Starting Index = 5

Ending Index = 8

Length = 4

Example

Input

8

9 8 7 6 5 4 3 2

Output

Longest Consecutive Increasing Sequence

9

Length = 1

Restrictions

- Do not sort the array.
- Do not use any additional array.
- Solve using a single traversal.

7: Smallest Missing Positive Integer

Write a program in Java to input N integers into an array. The array may contain positive numbers, negative numbers, and zero.

Determine the smallest positive integer that is not present in the array.

Sample Input 1

Enter number of elements: 8

3 4 -1 1 7 2 6 8

Output

Smallest Missing Positive Integer = 5

Sample Input 2

Enter number of elements: 5

1 2 3 4 5

Output

Smallest Missing Positive Integer = 6

Restrictions

- Do not sort the array.
- Do not use any Collection classes (ArrayList, HashSet, etc.).
- Do not use library search methods.
- Solve the problem using array traversal and logical reasoning.

8: Leaders in an Array

An element in an array is called a Leader if it is greater than every element to its right. The last element of the array is always considered a leader.

Write a Java program to input N integers into an array and display all the leader elements in the order in which they appear.

Sample Input

Enter number of elements: 8

16 17 4 3 5 2 8 1

Output

Leader Elements are:

17
8
1

Sample Input

Enter number of elements: 6

30 25 20 15 10 5

Output

Leader Elements are:

30
25
20
15
10
5

Restrictions

- Do not sort the array.
- Do not use another array to store the leaders.
- Display the leaders in their original order of appearance.
- Aim to solve the problem with a single traversal from right to left.

9. Without sorting,

Write a Program to create an array of size N, and find the second highest and second lowest value from it.

What are Multi-Dimensional Arrays in Java?

Multi-dimensional array is array of arrays, where each element of the main array holds another array. They are commonly used to represent data in table or matrix formats such as grids, charts or spreadsheets.

Multi-dimensional array can be compared to a table. For example, storing marks of students where each row represents the student and each column represents the subject. In this way data is organized in rows and columns using a multi - dimensional array.

Two-dimensional (2D) Array

A two dimensional array (2D) array is a type of multi - dimensional array that stores the data in the form of rows and columns. It is used to represent table - like structures such as matrices, grids or tables where data is organized in two dimensional only.

Declaration

You can use one of the following taxes to declare two dimensional array.

Syntax:

```
DataType[][] arrayname;  
Or  
DataType arrayName[][];
```

For memory allocation:

```
arrayName = new datatype[numberOfRows][numberOfColumns];
```

Or

Declaration along with memory allocation:

```
Datatype[][] arrayName = new datatype[numberOfRows][numberOfColumns];
```

Or

```
Datatype arrayName[][] = new datatype[numberOfRows][numberOfColumns];
```

Static variables window showing the state of a 2D array:

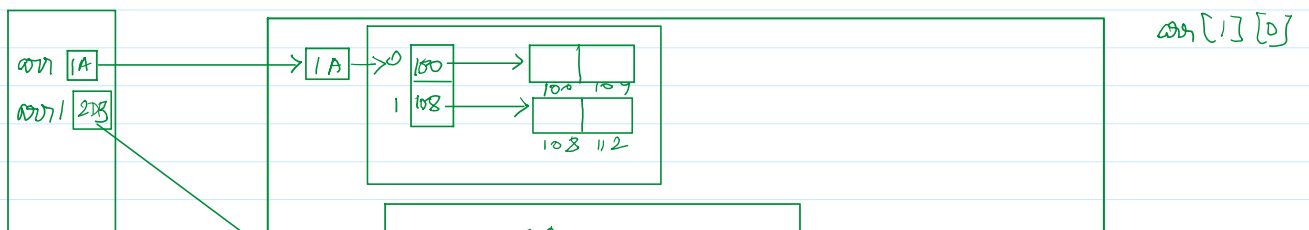
- int length: 2 (number of rows)
- [0]: 1, 2 (number of elements in first row)
- [1]: 2, 2

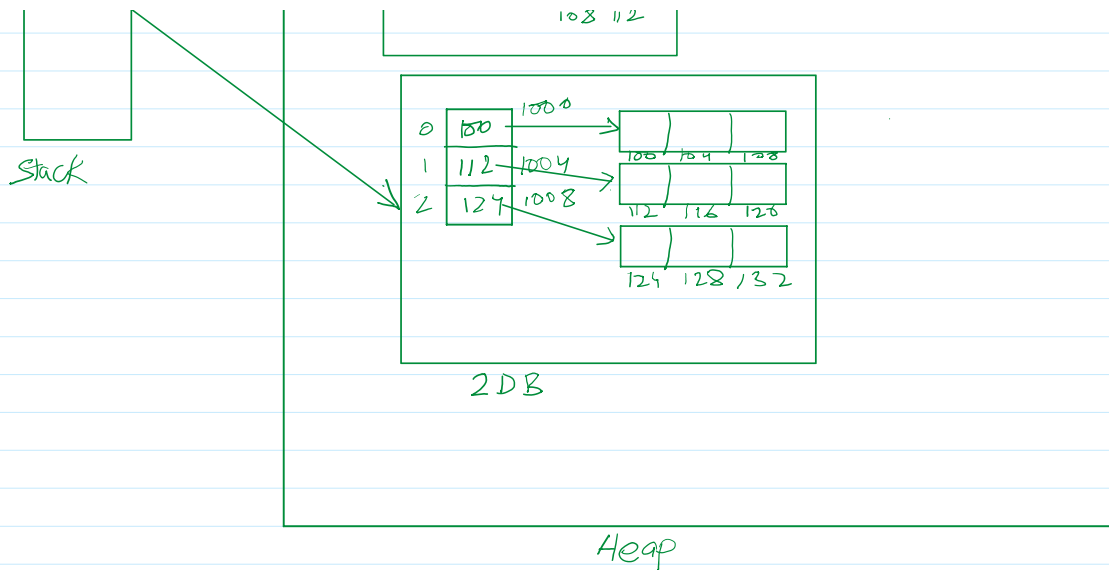
Static variables window showing the state of a 2D array:

- int length: 3
- [0]: 0, 0, 0
- [1]: 0, 0, 0
- [2]: 0, 0, 0

memory of 2D array is allocated by 'new' keyword dynamically during the execution of code, on the heap memory area.

default values assigned by JVM.





Default values assigned by Java compiler in array:

For integer type array: 0 (byte, short, int, long)

For floating point type: 0.0 (float, double)

For boolean: false

For char: '\u0000' or null

For String: null

For long type array:

Debugger window showing the state of a long array named 'arr2'. The array has a length of 3 and contains three elements, all of which are 0. The debugger window also shows the source code of the program, which includes the declaration and initialization of the array.

```

public class TwoArrayDemo
{
    public static Scanner sc = new Scanner(System.in);
    public static void main(String[] args){
        int[][] arr = {{1,2}, {3, 4}};

        int[][] arr1 = new int[3][3];
        long[][] arr2 = new long[3][3];
        char[][] arr3 = new char[3][3];
        String[][] arr4 = new String[3][3];

        float[][] arr5 = new float[3][3];
        double[][] arr6 = new double[3][3];

        boolean[][] arr7 = new boolean[3][3];
    }
}

```

For char[][] type:

Code:

```

7 public class TwoArrayDemo
8 {
9     public static Scanner sc = new Scanner(System.in);
10    public static void main(String[] args) {
11        int[][] arr = {{1,2}, {3, 4}};
12
13        int[][] arr1 = new int[3][3];
14        long[][] arr2 = new long[3][3];
15        char[][] arr3 = new char[3][3];
16        String[][] arr4 = new String[3][3];
17
18        float[][] arr5 = new float[3][3];
19        double[][] arr6 = new double[3][3];
20
21        boolean[][] arr7 = new boolean[3][3];
22    }
23 }

```

Debugger State (char arrays):

- Static fields:** int length: 3
- Instance variables:**
 - [0] : char[]**: int length: 3, [0]: '\u0000', [1]: '\u0000', [2]: '\u0000'
 - [1] : char[]**: int length: 3, [0]: '\u0000', [1]: '\u0000', [2]: '\u0000'
 - [2] : char[]**: int length: 3, [0]: '\u0000', [1]: '\u0000', [2]: '\u0000'

Local variables:

- String[] args = <object>
- int[][] arr = <object>
- int[][] arr1 = <object>
- long[][] arr2 = <object>
- char[][] arr3 = <object>
- String[][] arr4 = <object>
- float[][] arr5 = <object>
- double[][] arr6 = <object>
- boolean[][] arr7 = <object>

Default values for String[][]:

Code:

```

7 public class TwoArrayDemo
8 {
9     public static Scanner sc = new Scanner(System.in);
10    public static void main(String[] args) {
11        int[][] arr = {{1,2}, {3, 4}};
12
13        int[][] arr1 = new int[3][3];
14        long[][] arr2 = new long[3][3];
15        char[][] arr3 = new char[3][3];
16        String[][] arr4 = new String[3][3];
17
18        float[][] arr5 = new float[3][3];
19        double[][] arr6 = new double[3][3];
20
21        boolean[][] arr7 = new boolean[3][3];
22    }
23 }

```

Debugger State (String arrays):

- Static fields:** int length: 3
- Instance variables:**
 - [0] : String[]**: int length: 3, [0]: null, [1]: null, [2]: null
 - [1] : String[]**: int length: 3, [0]: null, [1]: null, [2]: null
 - [2] : String[]**: int length: 3, [0]: null, [1]: null, [2]: null

Local variables:

- String[] args = <object>
- int[][] arr = <object>
- int[][] arr1 = <object>
- long[][] arr2 = <object>
- char[][] arr3 = <object>
- String[][] arr4 = <object>
- float[][] arr5 = <object>
- double[][] arr6 = <object>
- boolean[][] arr7 = <object>

Default values for float[][] or double[][] is same:

```

7 public class TwoArrayDemo
8 {
9     public static Scanner sc = new Scanner(System.in);
10    public static void main(String[] args){
11        int[][] arr = {{1,2}, {3, 4}};
12
13        int[][] arr1 = new int[3][3];
14        long[][] arr2 = new long[3][3];
15        char[][] arr3 = new char[3][3];
16        String[][] arr4 = new String[3][3];
17
18        float[][] arr5 = new float[3][3];
19        double[][] arr6 = new double[3][3];
20
21        boolean[][] arr7 = new boolean[3][3];
22    }

```

Class compiled - no syntax errors

Default values for boolean[][] is false:

```

7 public class TwoArrayDemo
8 {
9     public static Scanner sc = new Scanner(System.in);
10    public static void main(String[] args){
11        int[][] arr = {{1,2}, {3, 4}};
12
13        int[][] arr1 = new int[3][3];
14        long[][] arr2 = new long[3][3];
15        char[][] arr3 = new char[3][3];
16        String[][] arr4 = new String[3][3];
17
18        float[][] arr5 = new float[3][3];
19        double[][] arr6 = new double[3][3];
20
21        boolean[][] arr7 = new boolean[3][3];
22    }

```

Class compiled - no syntax errors

The representation of 2D array

The following image shows the representation of 2D array.

columns

	0	1	2
0	0,0	0,1	0,2
1	1,0	1,1	1,2
2	2,0	2,1	2,2

rows

Two dimensional array in memory:

0,0	0,1	0,2	1,0	1,1	1,2	2,0	2,1	2,2
-----	-----	-----	-----	-----	-----	-----	-----	-----

Jagged arrays

The screenshot shows a Java IDE with the following code:

```

11 int[][] arr = {{1,2}, {3, 4}};
12
13 int[][] arr1 = new int[3][3];
14 long[][] arr2 = new long[3][3];
15 char[][] arr3 = new char[3][3];
16 String[][] arr4 = new String[3][3];
17
18 float[][] arr5 = new float[3][3];
19 double[][] arr6 = new double[3][3];
20
21 boolean[][] arr7 = new boolean[3][3];
22
23 int[][] arr8 = {{1,2}, {3}, {4,5,6}, {10},{2,4,6,8}};
24
25 }

```

Below the code, several windows display the memory layout of these arrays:

- arr (int[][]):** Shows a 5x5 grid of elements. The first row contains 1 and 2, and the rest are empty.
- arr1 (int[][]):** Shows a 3x3 grid of elements, all empty.
- arr2 (long[][]):** Shows a 3x3 grid of elements, all empty.
- arr3 (char[][]):** Shows a 3x3 grid of elements, all empty.
- arr4 (String[][]):** Shows a 3x3 grid of elements, all empty.
- arr5 (float[][]):** Shows a 3x3 grid of elements, all empty.
- arr6 (double[][]):** Shows a 3x3 grid of elements, all empty.
- arr7 (boolean[][]):** Shows a 3x3 grid of elements, all empty.
- arr8 (int[][]):** Shows a 4x4 grid of elements. The first row contains 1 and 2, the second row contains 3, the third row contains 4, 5, and 6, and the fourth row contains 10, 2, 4, 6, and 8.

```

int[][] arr8= {{1,2}, {3}, {4,5,6}, {10},{2,4,6,8}};
int i, j;
System.out.print("\fJagged Array:\n");
for(i = 0; i < arr8.length; i++, System.out.print("\n")){
    for(j = 0; j < arr8[i].length; j++){
        System.out.print("\t" + arr8[i][j]);
    }
}

```

number of rows (pointing to `arr8.length`)
number of columns (pointing to `arr8[i].length`)

Output:

```

Blue: Terminal Window - Kritin11
Options
Jagged Array:
    1      2
    3
    4      5      6
    10
    2      4      6      8

```

```

package MyArrays.TwoDArray;
import java.util.*;
/**
 * @author (your name)
 * @version (a version number or a date)
 */
public class TwoDArrayDemo
{
    public static Scanner sc = new Scanner(System.in);
    public static void main(String[] args){
        int[][] arr = {{1,2}, {3, 4}};
    }
}

```

```

int[][] arr1 = new int[3][3];
long[][] arr2 = new long[3][3];
char[][] arr3 = new char[3][3];
String[][] arr4 = new String[3][3];

float[][] arr5 = new float[3][3];
double[][] arr6 = new double[3][3];

boolean[][] arr7 = new boolean[3][3];

int[][] arr8= {{1,2}, {3}, {4,5,6}, {10},{2,4,6,8}};
int i, j;
System.out.print("\fJagged Array:\n");
System.out.print("\nTechnique-1\n");
for(i = 0; i < arr8.length; i++, System.out.print("\n")){
    for(j = 0; j < arr8[i].length; j++){
        System.out.print("\t" + arr8[i][j]);
    }
}

System.out.print("\nTechnique-2\n");
//For each loop only applicable for Collection (array list. ...)
for(int[] x: arr8){
    for(int y : x){
        System.out.print("\t"+y);
    }
    System.out.print("\n");
}

System.out.print("\nTechnique-3\n");
//For each loop only applicable for Collection (array list. ...)
for(int[] x: arr8){
    for(i=0; i < x.length; i++){
        System.out.print("\t"+x[i]);
    }
    System.out.print("\n");
}

System.out.print("\nTechnique-4");
//For each loop only applicable for Collection (array list. ...)
for(int[] x: arr8){
    System.out.print("\n"+java.util.Arrays.toString(x));
}

System.out.print("\nTechnique-5");
for(i = 0; i < arr8.length; i++){
    System.out.print("\n"+java.util.Arrays.toString(arr8[i]));
}
}
}

```

Output:

Jagged Array:

Technique-1

1 2

3

```
4 5 6
10
2 4 6 8
```

Technique-2

```
1 2
3
4 5 6
10
2 4 6 8
```

Technique-3

```
1 2
3
4 5 6
10
2 4 6 8
```

Technique-4

```
[1, 2]
[3]
[4, 5, 6]
[10]
[2, 4, 6, 8]
```

Technique-5

```
[1, 2]
[3]
[4, 5, 6]
[10]
[2, 4, 6, 8]
```

Three Dimensional (3D) Array (Not in ISC)

A 3 dimensional (3D) array is an array that contains arrays of arrays. It is used to store data in three dimensions such as layers, rows and columns. A 3D array is commonly used to represent complex data structures like a collection of matrices or data arranged across multiple levels.

It is a complex form of 2D array. In other words, it is an array of two dimensional array.

Declaration

Syntax:

```
DataType[][][] arrayName;
```

Or

```
DataType arrayName[][][];
```

For memory allocation:

```
arrayName = new Datatype[layers][numberOfRows][numberOfColumns];
```

Or

Declaration along with memory allocation

```
DataType[][][] arrayName = new DataType[layers][numberOfRows]
[numberOfcolumns];
```

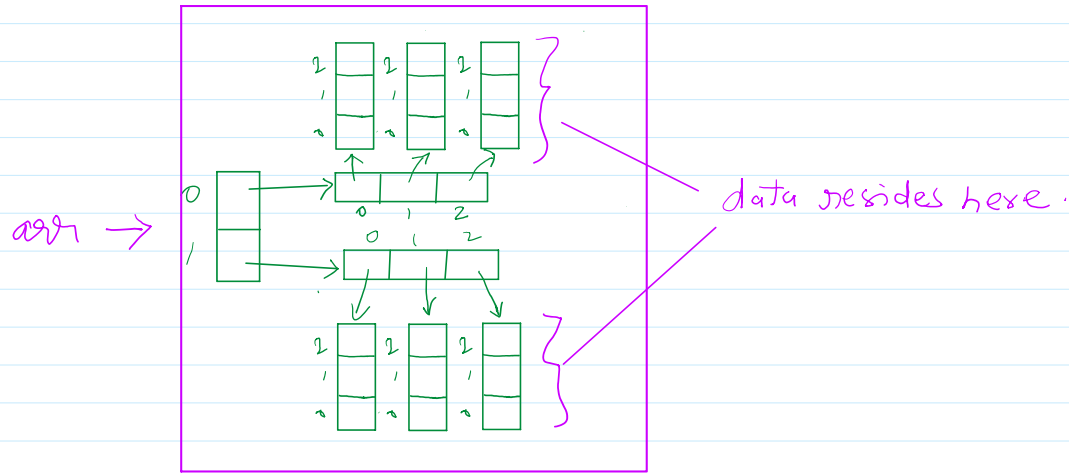
number of layers or 2D array

in each layer

Or

```
DataType arrayName[][][] = new DataType[numberOfLayers][numberOfRows][numberOfColumns];
```

```
int arr[][][] = new int[2][3][3];
```



Three dimensional Array.

```
package MyArrays.TwoDArray;  
import java.util.*;
```

```
/**  
 * Write a description of class ThreeDArrayDemo here.  
 *  
 * @author (your name)  
 * @version (a version number or a date)  
 */  
public class ThreeDArrayDemo  
{  
    public static void main(String[] args){  
        System.out.print("\f");  
        Random r = new Random();  
        int[][][] arr = new int[2][3][3];  
        int i, j, k;  
        for(i = 0; i < arr.length; i++){  
            System.out.print("\nLayer-"+i+"\n");  
            for(j = 0; j < arr[i].length; j++, System.out.print("\n")){  
                for(k = 0; k < arr[i][j].length; k++){  
                    arr[i][j][k] = r.nextInt(10, 100);  
                    System.out.print("\t" + arr[i][j][k] );  
                }  
            }  
        }  
    }  
}
```

Output:

Notes In Java array is the template of Arrays

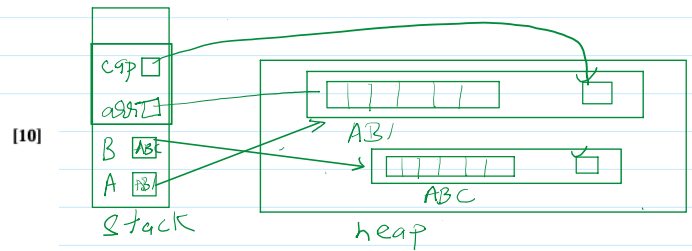
Output:

Blue: Terminal Window - Kritin11			
Options			
Layer-0			
70	48	55	
45	37	60	
94	76	96	
Layer-1			
56	95	53	
93	81	55	
60	94	41	

Notes In Java array is the template of Arrays class present in java.util package. Therefore, internally it assigns number of elements in the length variable, which is public instance variable in Arrays class.

array.length
↑

directly accessible because it is a public int variable in Arrays class.



Question 7

A class **Mixarray** contains an array of integer elements along with its capacity (More than or equal to 3). Using the following description, form a new array of integers which will contain only the first 3 elements of the two different arrays one after another.

Example: Array1: { 78, 90, 100, 45, 67 }
Array2: {10, 67, 200, 90 }
Resultant Array: { 78, 90, 100, 10, 67, 200}

The details of the members of the class are given below:

Class name : **Mixarray**

Data members/instance variables:

arr[] : integer array
cap : integer to store the capacity of the array

Member functions/methods:

→ Mixarray (int mm) ← constructor : to initialize the capacity of the array cap=mm
void input() : to accept the elements of the array
Mixarray mix(Mixarray P, Mixarray Q) : returns the resultant array having the first 3 elements of the array of objects P and Q
void display() : to display the array with an appropriate message.

Specify the class **Mixarray** giving details of the **constructor(int)**, **void input()**, **Mixarray mix(Mixarray, Mixarray)** and **void display()**. Define a **main()** function to create objects and call all the functions accordingly to enable the task. (Create)

```
public static void main(String[] args) {
    Mixarray A, B, C;
    int size1, size2;
    Sout ("Enter number of elements in A: ");
    A = new Mixarray(sc.nextInt());

    Sout ("Enter number of elements in B: ");
    B = new Mixarray(sc.nextInt());

    C = C.mix(A, B);
    sout ("Content of A: ");
    A.display();
    sout ("Content of B: ");
    B.display();
    sout ("Content of C: ");
    C.display();
}
```

```
package MyArrays;
import java.util.*;
```

```
/**
 * Write a description of class Mixarray here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Mixarray
{
    private int arr[], cap;

    private static Scanner sc = new Scanner(System.in);
```

```
private int arr[], cap;
```

```
private static Scanner sc = new Scanner(System.in);
```

```
public Mixarray(int mm){ ← constructor. ← it is called by 'new' keyword only,  
    cap = mm;                               while creating the object.  
    arr = new int[cap];  
}
```

```
public void input(){  
    int i;  
    for(i = 0; i < cap; i++){  
        System.out.print("Enter integer value in [" + i + "]: ");  
        arr[i] = sc.nextInt();  
    }  
}
```

```
public Mixarray mix(Mixarray P, Mixarray Q){  
    int i, j, np, nq;
```

```
    if(P.cap >= 3){  
        np = 3;
```

```
    }  
    else{  
        np = P.cap;
```

```
    }  
    if(Q.cap >= 3){  
        nq = 3;
```

```
    }  
    else{  
        nq = Q.cap;
```

```
    Mixarray m = new Mixarray(np+nq);
```

```
    for(i = 0; i < np; i++){  
        m.arr[i] = P.arr[i];  
    }
```

```
    for(i = 0 ; i < nq; i++){  
        m.arr[i+np] = Q.arr[i];  
    }
```

```
    return m;
```

```
}
```

```
public void display(){  
    System.out.print("{");  
    int i;  
    for(i = 0; i < cap - 1; i++){  
        System.out.print(arr[i]+", ");  
    }  
    System.out.print(arr[i]+"}\n");  
}
```

```
public static void main(String[] args){
```

```
    Mixarray A, B, C;
```

```
    System.out.print("\nEnter number of elements in A: ");
```

```
    A = new Mixarray(sc.nextInt());
```

```
    System.out.print("\nEnter number of elements in B: ");
```

creating (→


```

Mixarray A, B, C;
System.out.print("\nEnter number of elements in A: ");
A = new Mixarray(sc.nextInt());
System.out.print("Enter number of elements in B: ");
B = new Mixarray(sc.nextInt());
C = new Mixarray(6);
System.out.print("Enter values in first array: \n");
A.input();
System.out.print("Enter values in second array: \n");
B.input();
C = C.mix(A, B);
System.out.print("\nArray1: ");
A.display();
System.out.print("\nArray2: ");
B.display();
System.out.print("\nResultant Array: ");
C.display();
}
}

```

creating
objects

reference is passed to method

call by reference;

↑
we pass the reference of the object. This type of function may change the state of the object. This kind of function is called mutator function or impure function.

Home work:

Question 7

Design a class **OddEven** to arrange two single dimensional arrays into one single dimensional array, such that the odd numbers from both the arrays are at the beginning followed by the even numbers. [10]

Example: Array 1: { 2, 13, 6, 19, 26, 11, 4 }

Array 2: { 7, 22, 4, 17, 12, 45 }

Arranged Array = { 13, 19, 11, 7, 17, 45, 2, 6, 26, 4, 22, 4, 12 }

Some of the members of the class are given below:

Class name : **OddEven**

Data members/instance variables:

a[] : to store integers in the array
m : integer to store the size of the array

Methods / Member functions:

OddEven(int mm) : parameterised constructor to initialize the data member m=mm
void fillarray() : to enter integer elements in the array
OddEven arrange(OddEven P, OddEven Q) : stores the odd numbers from both the parameterized object arrays followed by the even numbers from both the arrays and returns the object with the arranged array
void display() : displays the elements of the arranged array

Specify the class **OddEven** giving details of the **constructor()**, **void fillarray()**, **OddEven arrange(OddEven, OddEven)** and **void display()**. Define a **main()** function to create objects and call the functions accordingly to enable the task.

Question 7

A class **Composite** contains a two-dimensional array of order $[m \times n]$. The maximum values possible for both 'm' and 'n' is 20. Design a class **Composite** to fill the array with the first $(m \times n)$ composite numbers in column wise. [Composite numbers are those which have more than two factors.] [10]

The details of the members of the class are given below:

Class name : **Composite**

Data members/instance variables:

arr[][] : integer array to store the composite numbers
column wise
m : integer to store the number of rows
n : integer to store the number of columns

Member functions/methods:

Composite(int mm, int nn) : to initialize the size of the matrix, $m=mm$ and $n=nn$
int isComposite(int p) : to return 1 if the number is composite otherwise returns 0
void fill () : to fill the elements of the array with the first $(m \times n)$ composite numbers in column wise
void display() : to display the array in a matrix form

Specify the class **Composite** giving details of the **constructor(int,int)**, **int isComposite(int)**, **void fill()** and **void display()**. Define a **main()** function to create an object and call all the functions accordingly to enable the task.

Question 9:

Write a function `int len(String s)` that returns the length of the String `s` using a recursive technique. The main function need not be written. [5]

Question 10:

Design a class **Symmetric** that checks whether a square matrix is symmetric or not. A square matrix is **Symmetric** if it is equal to its transpose. [5]

For example:

2	3	6
3	4	5
6	5	9

 is a Symmetric matrix

Class name: **Symmetric**

Data Members/Instance variables

arr[][] : integer array to store the elements
m : stores the order of the matrix

Symmetric(int xx) – parameterized constructor to initialize `m` with `xx` and the array with default values

void fill() – accepts values into the array `arr`.

Question 7:

[10]

A class `fill_fibo` contains a two dimensional array of order $m \times n$. Design a class `fill_fibo` to fill the array with the first $m \times n$ numbers of the Fibonacci series row wise.
A Fibonacci Series is a series of numbers in which each number is the sum of the two preceding numbers. The series is as follows 0 1 1 2 3 5.....

Class name: `fill_fibo`

Data Members/ Instance variables

`arr[][]` - integer array to store the Fibonacci series numbers row-wise
`m` - integer to store the number of rows
`n` - integer to store the number of columns

Member functions / Methods :

`fill_fibo(int r,int c)` : to initialize the size of the matrix $m=r$ and $n=c$
`int isFibonacci(int p)` : returns 1 if the number is a part of the Fibonacci series, else returns 0
`void fill()` : to fill the elements of the array with the first $(m \times n)$ Fibonacci numbers in row wise
`void display()` : to display the array in matrix form.

Specify the class `fill_fibo` giving details of the constructor(`int,int`), `int isFibonacci(int)`, `void fill()` and `void display()`. Define a `main()` function to create an object and call all the functions accordingly to enable the task

Question 8:

[10]

A class `new_str` replaces each character of a word by the characters succeeding it or preceding it depending on the number provided as an argument.

For example if the word is "HELLO" and the integer number is 5, the new word formed will be "MJQQT", as all the characters of the word move 5 steps forward.

However if the word is "ABCDE" and the number is -5, the new word formed will be "VWXYZ", as all the characters move 5 steps backward.

However, the arrangement of the letters should be performed in a circular manner, i.e. Z should follow A in the backward direction and A should follow Z in the forward direction.

Question 7

Write a program that takes a 2D binary matrix of M rows and N columns to check whether the matrix is horizontal symmetric, vertical symmetric, or both.

The matrix is said to be horizontal symmetric if the first row is the same as the last row, the second row is the same as the second last row, and so on.

And the matrix is said to be vertical symmetric if the first column is the same as the last column, the second column is the same as the second last column, and so on.

Print "VERTICAL" if the matrix is vertically symmetric, "HORIZONTAL" if the matrix is horizontally symmetric, "BOTH" if the matrix is vertical and horizontal symmetric, and "NOT SYMMETRIC" if not symmetric.

Examples:**Input:**

0 1 0
0 0 0
0 1 0

Create a class `Symmetric` that fills up a two dimensional matrix of order ' $m \times n$ ' and checks whether it is a symmetric matrix or not. Display the original array and an appropriate message displaying whether the matrix is a horizontally symmetric, vertically symmetric, both way symmetric or not symmetric

Data Members

`arr[][]`: two dimensional integer array
`m`: number of rows
`n`: number of columns

Member Functions

`Symmetric (...)`: Parameterized constructor to initialize data members

`void Input()` : Inputs array elements

<code>boolean IsHorizontal()</code>	Returns true if the matrix is horizontally symmetric, otherwise false
<code>boolean IsVertical()</code>	Returns true if the matrix is vertically symmetric, otherwise false
<code>void Display():</code>	Displays the original matrix and an appropriate message displaying whether the matrix is a horizontally symmetric, vertically symmetric, both way symmetric or not symmetric

Specify the class Symmetric giving the details of the constructor (), and other member functions. Define a main () function to create an object and call the other functions to enable the task.